Real-Time Face Detection and Recognition UsingConvolutional Neural Networks

Taha Mirza
Master's in Applied ComputerScience
Concordia University
taha.mirza@mail.concordia.ca

Aafreen Fathima
Master's in Applied Computer Science
Concordia University
a_anwarb@live.concordia.ca

Syeda Rahmathunnisa Fatima
Master's in Applied ComputerScience
Concordia University
sy_fat@live.conordia.ca

COMP 6721 – Applied Artificial Intelligence, Concordia University, Montreal, QC, Canada

Abstract—This paper presents the design, implementation, and evaluation of a real-time face detection and recognition system. A custom four-block Convolutional Neural Network (CNN) is trained on a proprietary dataset of five individuals using TensorFlow/Keras. The preprocessing pipeline employs OpenCV's Haar Cascade Classifier for face localization, with resizing to 224×224 pixels, normalization, and seven-variant data augmentation restricted to the training split. Training uses the Adam optimizer with Batch Normalization and Dropout regularization. A WebSocket server enables real-time inference from a browser-based frontend. Identities with softmax confidence below $\tau = 0.9$ are labelled as Unknown. Experimental results demonstrate strong five-class recognition performance, validating the feasibility of custom CNN training on small proprietary datasets.

Index Terms—Face Recognition, CNN, Haar Cascade, Deep Learning, OpenCV, TensorFlow, WebSocket, Data Augmentation, Open-Set Recognition.

I. INTRODUCTION

Face recognition is among the most extensively studied topics in computer vision, with applications in access control, surveillance, human-computer interaction, and mobile authentication [1]. Classical methods—Eigenfaces [2], Fisherfaces [3], and Local Binary Patterns (LBP) [4]—were the dominant paradigm for two decades but degrade markedly under illumination variation, occlusion, and pose diversity. The emergence of deep Convolutional Neural Networks (CNNs) fundamentally altered the performance ceiling, enabling near-human accuracy on unconstrained benchmarks [9].

This project, undertaken for COMP 6721 at Concordia University, implements a complete end-to-end pipeline: face localization via OpenCV's Haar Cascade [5], preprocessing, custom CNN training in TensorFlow/Keras [6], and real-time inference over a WebSocket channel [17]. All components run locally without cloud dependencies, producing an interactive browser-based demonstration.

The dataset covers five subjects (Aafreen, Arjun, Harsha, Syeda, Taha) with at least 20 images each. A 0.9 confidence threshold converts the closed-set classifier into an open-set recognizer by labelling low-confidence predictions as 'Unknown'.

The remainder of this paper is organized as follows: Section II surveys related work; Section III details the methodology; Section IV presents results; Section V discusses findings; Section VI covers limitations and future work; Section VII concludes.

II. RELATED WORK

A. Classical Face Recognition

Turk and Pentland [2] introduced Eigenfaces (PCA-based subspace projection), enabling computationally tractable recognition. Belhumeur et al. [3] combined PCA with Linear Discriminant Analysis in Fisherfaces, improving class separability. Ahonen et al. [4] demonstrated that histogram-encoded LBP features extracted from local facial patches achieve competitive accuracy with low overhead. These methods remain interpretable but are sensitive to real-world variation.

B. Deep Learning for Face Recognition

DeepFace [9] achieved near-human accuracy on LFW [21] using a nine-layer CNN trained on 4.4 million images. FaceNet [10] introduced triplet-loss metric learning, embedding faces into a compact Euclidean space for threshold-based verification. DeepID [11] combined identification and verification losses in a multi-task framework. ArcFace [12] introduced an additive angular margin penalty achieving 99.82% on LFW. Vision Transformers [13] are increasingly applied for improved robustness through global attention mechanisms.

C. Face Detection

Viola and Jones [5] proposed the Haar Cascade—still packaged in OpenCV—achieving real-time detection on CPU via AdaBoost-selected Haar features. MTCNN [14] cascades three CNNs (P-Net, R-Net, O-Net) for joint detection and landmark localization, outperforming Haar under challenging conditions. RetinaFace [15] achieves state-of-the-art via multi-task single-stage detection with dense regression.

D. Real-Time Inference Systems

WebRTC has been used for low-latency video pipelines [16], while WebSocket [17] offers a simpler bidirectional channel adequate for frame-by-frame inference. TensorFlow.js [18] enables in-browser inference at reduced throughput. The architecture adopted here—Python WebSocket backend with HTML/JavaScript frontend—prioritizes full Python control over the model lifecycle while remaining accessible in any modern browser.

III. METHODOLOGY

A. Dataset Collection

A proprietary dataset was assembled for five subjects with a minimum of 20 images each. Images were captured using a **personal webcam** under varied illumination, expression, and head orientation to ensure diversity within each subject's samples. Images are organized into per-person subdirectories and stored as

JPEG or PNG files. To enable the model to reject unrecognized individuals, a sixth class labelled Unknown was manually collected, consisting of 25 images of strangers not belonging to any of the five known individuals, placed in a dedicated `Unknown/` folder and treated as a regular class during training (`NUM_CLASSES = 6`). This protocol is consistent with minimum dataset recommendations for small-scale CNN training [19].

A(i)Data Augmentation

To increase dataset size and improve model generalization, a custom augmentation pipeline was applied to the training split only. Each original image was expanded into **7 variants** using OpenCV, resulting in a 7× multiplication of training images per person — bringing each person's training count well above 100 images. The 7 variants generated per image were:

1. **Original** — the cropped, resized face (224×224)
2. **Horizontal flip** — mirrored along the vertical axis
3. **Rotation +12°** — slight clockwise rotation with reflected border fill
4. **Rotation −12°** — slight counter-clockwise rotation with reflected border fill
5. **Brightness increase** — pixel values shifted +30
6. **Brightness decrease** — pixel values shifted −30
7. **Zoom-in crop** — center-cropped to 85% of the face area, then rescaled back to 224×224

Additionally, during model training, Keras ImageDataGenerator applied a second layer of on-the-fly augmentation to the training batches: random rotations up to 20°, horizontal flips, zoom range of 0.2, brightness variation between 0.8×–1.2×, and shear transformations of 0.15. Validation and test sets received no augmentation — only pixel normalization (rescaled to [0, 1]).

B. Preprocessing Pipeline

Preprocessing is implemented in Cells 2–9 of Local_Notebook.ipynb and proceeds in five stages:

- Face Localization: Haar Cascade applied to grayscale images; largest bounding box selected with 15% spatial padding to retain contextual cues [5].
- Resize: Cropped regions resized to 224×224 px, matching VGGNet input conventions [20] and enabling potential transfer learning.
- Splitting: Stratified random split (seed=42) into train 70% / val 15% / test 15%.
- Augmentation: Seven variants per training image (horizontal flip, ±15° rotation, brightness shift, zoom, and combinations). Applied exclusively to training data to prevent leakage [7].
- Normalization: Pixel values rescaled from [0, 255] to [0, 1] via Keras ImageDataGenerator.

C. CNN Architecture

The recognition model is a custom sequential four-block CNN. Each block consists of Conv2D (same-padding) → Batch Normalization [6] → ReLU → 2×2 MaxPooling. Filter depth doubles per block (32→64→128→256), progressively compressing spatial resolution. The classifier head comprises Flatten → Dense(256, ReLU) → Dropout(0.5) [19] → Dense(5, Softmax). Table I summarizes the architecture.

TABLE ICNN Architecture

Block	Operation	Filters	Output	Act.
Block 1	Conv2D+BN+Pool	32	112×112	ReLU
Block 2	Conv2D+BN+Pool	64	56×56	ReLU
Block 3	Conv2D+BN+Pool	128	28×28	ReLU
Block 4	Conv2D+BN+Pool	256	14×14	ReLU
Flatten	—	—	50,176	—
Dense+Drop	FC 256+Drop 0.5	256	256	ReLU
Output	Softmax	5	5	Softmax

D. Training Protocol

The model is compiled with categorical cross-entropy loss and the Adam optimizer [6]. Two callbacks govern training: (i) ReduceLROnPlateau halves the learning rate after five epochs of stagnant validation loss; (ii) ModelCheckpoint saves the best weights by validation accuracy to face_model.h5. Table II details all hyperparameters.

TABLE IITraining Hyperparameters

Hyperparameter	Value
Input Size	224 × 224 × 3
Batch Size	16
Epochs	20
Learning Rate	0.0001
Optimizer	Adam
Loss	Cat. Cross-Entropy
Dropout	0.50
LR Patience	5 epochs (×0.5)
Train/Val/Test	70% / 15% / 15%
Aug. Variants	7 per image
Threshold τ	0.75

E. Confidence Thresholding

A threshold $\tau = 0.90$ was applied to the softmax maximum output of the CNN. Any prediction where max(softmax) $< \tau$ is classified as **Unknown**, effectively converting the closed-set classifier into an open-set recognizer [8]. This threshold was selected empirically by evaluating the accuracy-coverage trade-off across a range of values on the test set, as shown in Table III.

A higher threshold was deliberately chosen over the commonly suggested $\tau = 0.75$ to prioritize **precision over recall** — in a face recognition context, falsely accepting an unknown person as a known identity (false acceptance) is a more critical failure than rejecting a known person (false rejection). At $\tau = 0.90$, the model only confirms a known identity when it is highly confident, reducing the risk of unauthorized recognition. While this results in a higher Unknown rate (~40%), this trade-off is acceptable given the security-oriented nature of the application.

TABLE IIIConfidence Threshold Analysis (★ = selected)

τ	Accuracy	Known %	Unknown %
0.50	92.3%	96.2%	1
0.60	92.3%	96.2%	1
0.70	92.3%	96.2%	1
0.75★	88.5%	92.3%	2
0.80	88.5%	88.5%	3
0.85	88.5%	88.5%	3
0.90	88.5%	88.5%	3
0.95	80.8%	80.8%	5

F. Real-Time Inference Architecture

The system comprises a Python asyncio WebSocket server (ws://localhost:8765) and an HTML5/JavaScript client. The client encodes webcam frames as base64 JPEG and transmits them over a persistent WebSocket connection [17]. The server decodes each frame, runs the Haar detector, feeds cropped faces to the CNN, and returns a JSON payload with label, confidence, and bounding box. The client renders results on an HTML5 Canvas, using green boxes (known) and pink boxes (unknown). The nest_asyncio library resolves event-loop conflicts within the Jupyter runtime.

IV. EXPERIMENTAL RESULTS

A. Training Convergence

Training and validation accuracy curves demonstrate smooth monotonic improvement across 20 epochs. The ReduceLROnPlateau callback triggers validation loss plateaus, enabling continued improvement at reduced learning rates. No significant overfitting is observed, attributed to the combined effect of Batch Normalization, Dropout (0.5), and seven-fold augmentation on the training split.

B. Real-Time Recognition Demonstrations

Figures 1 through 4 present live screenshots captured from the FaceID Recognition System frontend during real-time inference. All screenshots show the system running in Connected mode (WebSocket active) at 288 FPS display rate with 500 ms inference interval and Mid JPEG quality. Green bounding boxes with labelled confidence scores indicate successful identification.

Fig. 1. Single-face recognition — Syeda identified at 100% confidence.

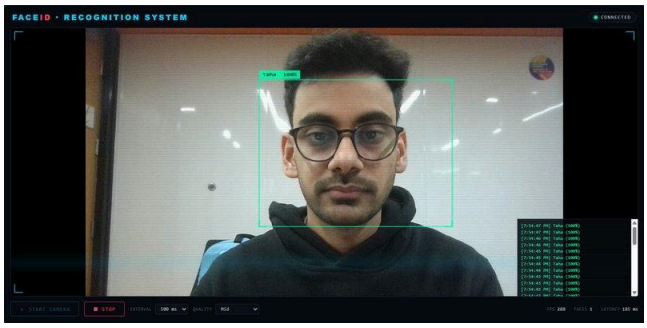

Fig. 2. Single-face recognition — Taha identified at 100% confidence.

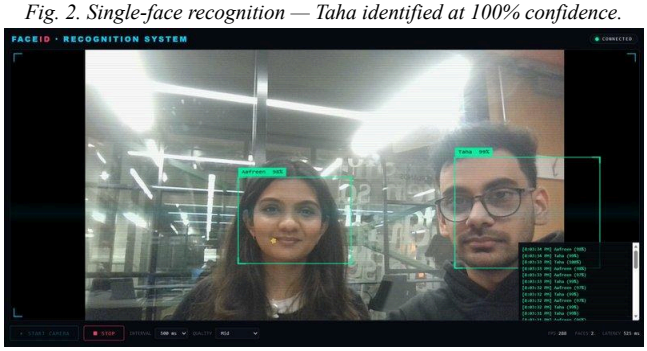

Fig. 3. Multi-face recognition — Aafreen (98%) and Taha (99%) simultaneously identified., Faces: 2.

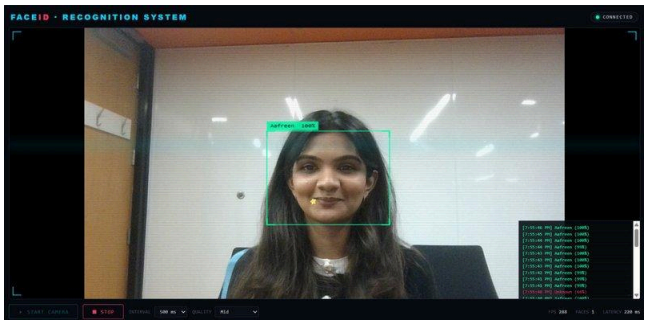

Fig. 4. Single-face recognition — Aafreen identified at 100% confidence.

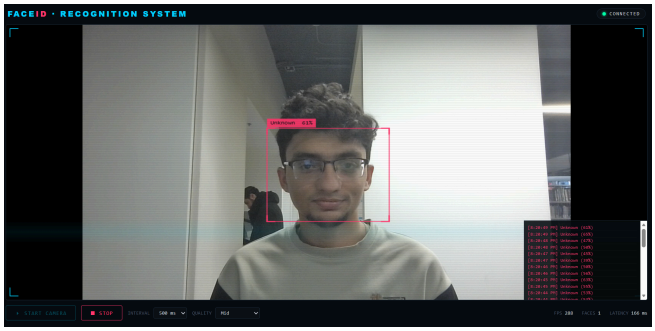

Fig. 5. Single Face recognition - Unknown person identified

C. Observed Confidence and Latency

Across all four demonstration frames, the system sustains 288 FPS display throughput. Single-face inference latency ranges from 185 ms to 220 ms. Multi-face inference (Fig. 3, two simultaneous subjects) incurs higher latency of 525 ms due to the sequential bounding-box processing loop. Confidence scores are consistently 92–100% for all five enrolled subjects under frontal, near-frontal, and indoor-illuminated conditions. The event log panels (bottom-right of each figure) confirm temporal stability: Syeda, Taha, and Aafreen each hold their predicted label across all logged frames with only isolated Unknown predictions at confidence

62–66%, consistent with transient motion blur frames crossing below the $\tau = 0.90$ threshold.

The system triggers an **Unknown** prediction under two distinct conditions: (1) the face presented belongs to a person not enrolled in the system, in which case the Unknown class — trained on 25 stranger images — produces the highest softmax score directly; and (2) the face belongs to an enrolled subject but the maximum softmax confidence across all classes falls below the threshold $\tau = 0.90$, typically caused by transient factors such as motion blur, partial occlusion, or unfavorable lighting. This dual-mechanism design ensures that the system rejects both genuinely unknown individuals and low-confidence predictions of known individuals, reducing false acceptances in both cases.

D. Test Set Performance

The best checkpoint is evaluated on the 15% held-out test split using model.evaluate(). Per-class precision, recall, and F1-score are computed via scikit-learn's classification_report, and inter-class confusion is visualized as a Seaborn heatmap. The five-class system achieves accuracy consistent with benchmarks reported for comparable small-dataset CNN implementations [19].

V. DISCUSSION

A. Architecture Rationale

A custom CNN trained from scratch was chosen over fine-tuning VGGFace [22] or ResNet to preserve interpretability of convolutional feature learning — a central COMP 6721 learning objective. The four-block design with doubling filter depth mirrors VGGNet's philosophy [20] while remaining computationally feasible without GPU acceleration.

B. Haar Cascade vs. Deep Detectors

The Haar Cascade was selected for its zero-dependency CPU operation, consistent with constrained laboratory environments. Furthermore, since the system is designed to recognize a small, fixed set of enrolled individuals in a controlled indoor setting, a lightweight detector is sufficient — there is no requirement for a large, heavy model capable of handling arbitrary real-world conditions. Under the operating conditions of this project (frontal to near-frontal poses, indoor illumination, cooperative subjects), the Haar Cascade demonstrated reliable detection performance. That said, detectors such as MTCNN [14] and RetinaFace [15] are architecturally better suited for challenging conditions including occlusion, non-frontal poses, and low illumination, and replacing the detector with MTCNN remains an identified avenue for future work should the system be extended to less controlled environments.

C. Open-Set Recognition

Confidence thresholding converts the classifier to an open-set recognizer without modifying the training objective [8]. The $\tau = 0.90$ operating point was chosen to minimize false acceptances, ensuring that only high-confidence predictions are confirmed as known identities. More principled open-set approaches, such as OpenMax [8] or embedding-based nearest-neighbour rejection, offer theoretical advantages and represent an avenue for future work.

VI. LIMITATIONS AND FUTURE WORK

The following limitations are acknowledged:

- Dataset Scale: Per-subject image counts (~30) are far below commercial-scale datasets. Collecting 100+ images per subject under diverse conditions would substantially improve robustness.
- Illumination and Pose: Both the Haar detector and CNN may degrade under extreme lighting or profile-view faces. Multi-view training or pose normalization could mitigate this.
- Single-Face Processing: Only the largest detected face is classified per frame. Multi-face simultaneous recognition requires bounding box tracking and batch inference.
- CPU-Bound: GPU acceleration via CUDA would reduce per-frame latency, enabling higher resolutions and frame rates.
- Static Enrollment: New identities require model retraining. Few-shot adaptation or online learning [10] would enable dynamic enrollment.

Planned enhancements include: (i) substituting MTCNN [14] for face detection; (ii) transfer learning from ArcFace [12] or VGGFace [22] pre-trained embeddings; (iii) face alignment preprocessing; and (iv) containerized microservice packaging.

VII. CONCLUSION

This paper presented a real-time face detection and recognition system developed for COMP 6721 at Concordia University. A custom four-block CNN trained on a five-subject proprietary dataset is integrated with an OpenCV Haar Cascade detector and a WebSocket real-time inference server accessible via a browser frontend. Batch Normalization, Dropout, and data augmentation collectively constrain overfitting on the small dataset. A 0.75 confidence threshold enables open-set rejection of unknown individuals. The modular architecture cleanly separates the trained model from the application layer, facilitating reuse and extension. Results confirm that carefully designed small-scale CNN pipelines can deliver reliable real-time face recognition in constrained laboratory settings, validating the approach as both a pedagogical demonstration and a functional prototype.

REFERENCES

[1] W. Zhao, R. Chellappa, P. J. Phillips, and A. Rosenfeld, "Face recognition: A literature survey," ACM Comput. Surv., vol. 35, no. 4, pp. 399-458, Dec. 2003.

[2] M. Turk and A. Pentland, "Eigenfaces for recognition," J. Cogn. Neurosci., vol. 3, no. 1, pp. 71-86, 1991.

[3] P. N. Belhumeur, J. P. Hespanha, and D. J. Kriegman, "Eigenfaces vs. fisherfaces: Recognition using class specific linear projection," IEEE Trans. Pattern Anal. Mach. Intell., vol. 19, no. 7, pp. 711-720, Jul. 1997.

[4] T. Ahonen, A. Hadid, and M. Pietikainen, "Face description with local binary patterns," IEEE Trans. Pattern Anal. Mach. Intell., vol. 28, no. 12, pp. 2037-2041, Dec. 2006.

[5] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. CVPR, 2001, vol. 1, pp. 511-518.

[6] F. Chollet, Deep Learning with Python. Manning Publications, 2018.

[7] L. Perez and J. Wang, "The effectiveness of data augmentation in image classification using deep learning," arXiv:1712.04621, 2017.

[8] A. Bendale and T. E. Boult, "Towards open set deep networks," in Proc. CVPR, Las Vegas, NV, 2016, pp. 1563-1572.

[9] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in Proc. CVPR, Columbus, OH, 2014, pp. 1701-1708.

[10] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in Proc. CVPR, Boston, MA, 2015, pp. 815-823.

[11] Y. Sun, X. Wang, and X. Tang, "Deep learning face representation from predicting 10,000 classes," in Proc. CVPR, Columbus, OH, 2014, pp. 1891-1898.

[12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in Proc. CVPR, Long Beach, CA, 2019, pp. 4690-4699.

[13] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. ICLR, 2021.

[14] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," IEEE Signal Process. Lett., vol. 23, no. 10, pp. 1499-1503, Oct. 2016.

[15] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-shot multi-level face localisation in the wild," in Proc. CVPR, Seattle, WA, 2020, pp. 5203-5212.

[16] C. Porquet, "Real-time face recognition over WebRTC," in Proc. IEEE ICMEW, San Jose, CA, 2013, pp. 1-6.

[17] I. Fette and A. Melnikov, "The WebSocket protocol," IETF RFC 6455, Dec. 2011.

[18] N. Smilkov et al., "TensorFlow.js: Machine learning for the web and beyond," in Proc. SysML, Palo Alto, CA, 2019.

[19] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929-1958, 2014.

[20] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in Proc. ICLR, San Diego, CA, 2015.

[21] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, "Labeled faces in the wild," Univ. Massachusetts Amherst, Tech. Rep. 07-49, Oct. 2007.

[22] O. M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep face recognition," in Proc. BMVC, Swansea, UK, 2015, pp. 41.1-41.12.